Parallel Coordinates & Radar Charts

Workshop 5 — Module 2

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

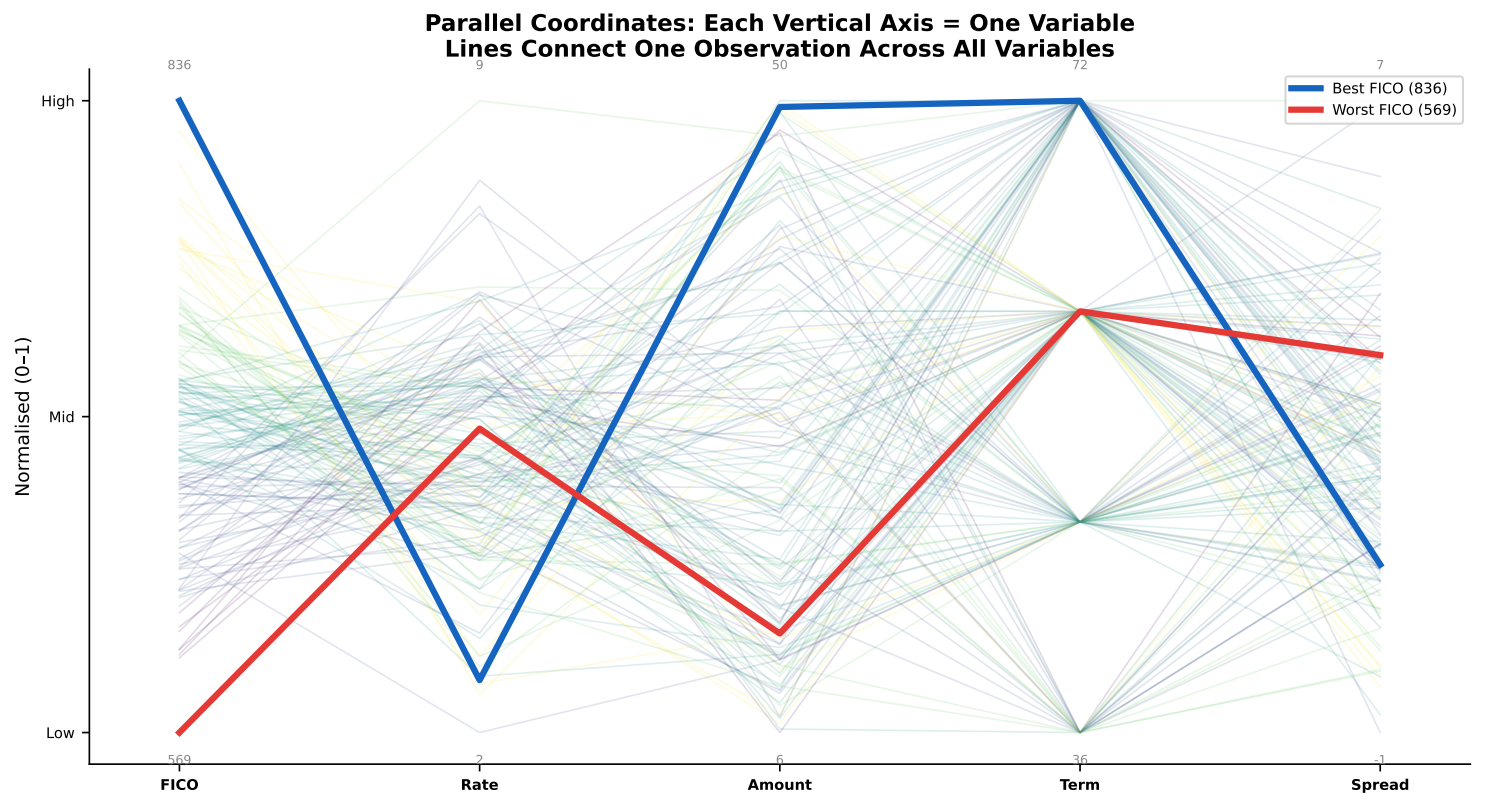
1. Parallel coordinates: each axis = one variable
2. Reading patterns: clusters, crossings, and outliers
3. Best practices: axis order, colour, highlighting
4. Radar/spider charts: when they work ($\leq$3 profiles)
5. When radar fails: area distortion and spaghetti
6. Choosing: parallel coordinates vs radar vs pairs plot

Learning Outcome

You will produce parallel coordinate plots for 5–20 variables with colour-grouped highlighting, radar charts for profile comparisons ($\leq$3 entities), and correctly choose between the two based on the number of profiles and variables.

Parallel Coordinates

Each Axis = One Variable



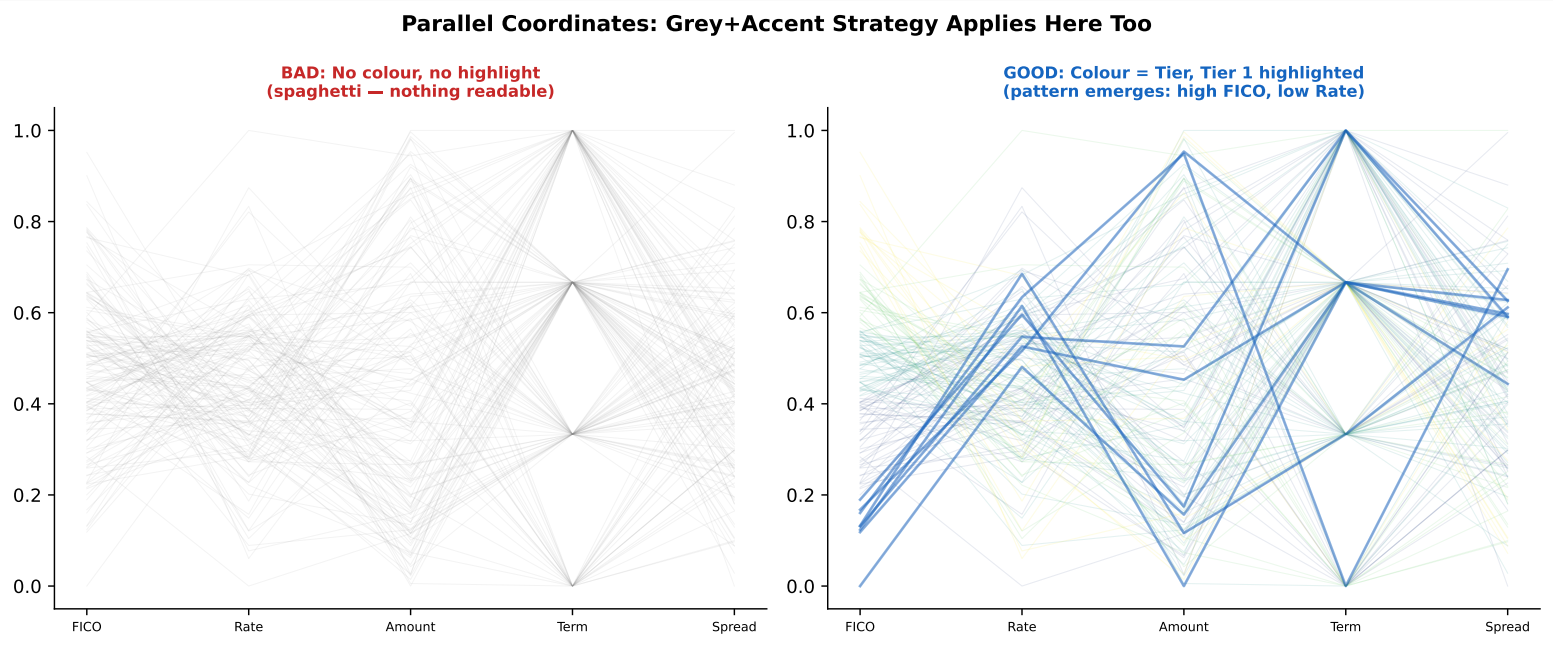
How to Read Parallel Coordinates

Pattern	Meaning
Parallel lines (same slope between axes)	Positive correlation between tho
Crossing lines (X pattern)	Negative correlation — values f
Cluster of lines (same colour, similar path)	A group with similar multivariat
Single line far from the cluster	Multivariate outlier
Bottleneck (all lines pass through one point)	That variable has low vari constant)

Pro-Tip

Adjacent axes are compared most easily (Gestalt proximity). Place the two variables you most want to compare **next to each other**. Axis reordering is the single most impactful design decision in parallel coordinates.

Best Practices: Grey + Accent



Parallel Coordinates in Code

R (GGally)

```r
library(GGally)

ggparcoord(ecar,
  columns = c("FICO", "Rate",
    "Amount", "Term", "Spread"),
  groupColumn = "Tier",
  scale = "uniminmax",  # 0-1
  alphaLines = 0.15,
  showPoints = FALSE,
  order = "anyClass") + # reorder
  scale_color_viridis_d() +
  theme_minimal() +
  labs(title = "Parallel Coords",
       color = "Tier")

# Highlight one group
ggparcoord(ecar,
  columns = 1:5,
  groupColumn = "highlight",
  alphaLines = 0.05) +
  scale_color_manual(values =
    c("TRUE"="#E53935",
      "FALSE"="#CCCCCC"))
```

Python (manual or pandas)

```python
from sklearn.preprocessing import
      (
    MinMaxScaler)

cols = ["FICO","Rate","Amount",
        "Term","Spread"]
scaler = MinMaxScaler()
normed = scaler.fit_transform(
    df[cols])

fig, ax = plt.subplots(figsize
     =(10,5))
for i in range(min(200, len(df)))
     :
    ax.plot(range(len(cols)),
        normed[i],
        color=cmap(norm(df["Tier"
            ][i])),
        alpha=0.12, lw=0.6)

ax.set_xticks(range(len(cols)))
ax.set_xticklabels(cols)

# pandas built-in (simpler)
```

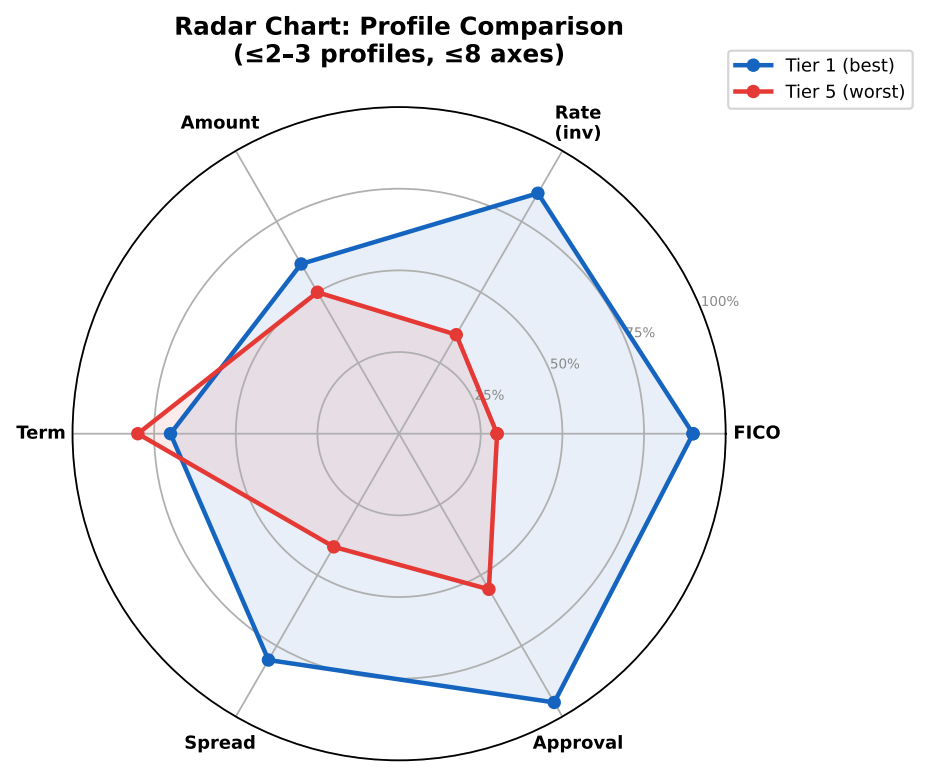
Axis Reordering: The Key Design Decision

Strategy	When to Use
Domain order	When variables have a natural sequence (e.g., time steps, pipeline stages)
Correlation order	Place highly correlated pairs adjacent to reveal parallel/crossing patterns
Variance order	Place highest-variance axes at edges for visual anchoring
Algorithmic (clusterpath)	`ggparcoord(order="anyClass")` finds the order that best separates groups

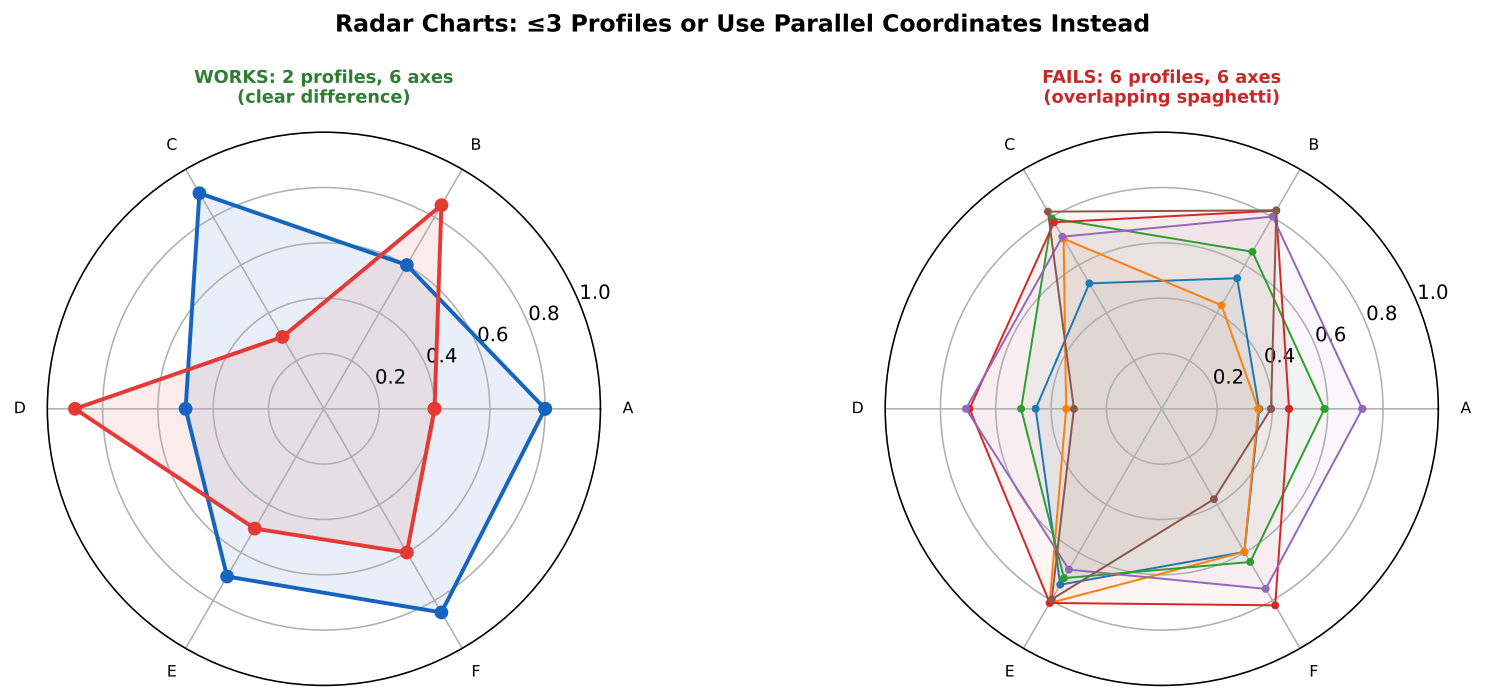
Try at least 2–3 orderings before settling. The "right" order depends on the question you are investigating.

Radar Charts

Radar: Profile Comparison ($\leq$3 Profiles)



When Radar Fails



Pro-Tip

Radar charts have three known problems: (1) **area is misleading** — the enclosed area depends on axis order, not just the values. (2) **Axis order is arbitrary** for non-sequential variables, so the shape is meaningless. (3) Beyond 3 profiles, they become spaghetti. Use radar only for **2–3 profiles with $\leq$8 axes** where the goal is holistic comparison, not precise value reading.

Radar in Code

R (ggradar or manual)

```r
# ggradar (simple)
# install.packages("ggradar")
library(ggradar)
ggradar(profiles,
  grid.min = 0, grid.max = 1,
  group.line.width = 1.5,
  group.point.size = 3,
  fill = TRUE, fill.alpha = 0.1,
  legend.position = "bottom")

# Manual with coord_polar
ggplot(df_long, aes(x = variable,
    y = value, group = profile,
    color = profile)) +
  geom_polygon(fill = NA,
    linewidth = 1.2) +
  geom_point(size = 2) +
  coord_polar() +
  scale_color_manual(values =
    c("#1565C0","#E53935")) +
  theme_minimal()
```

Python (manual polar)

```python
categories = ["FICO","Rate",
    "Amount","Term","Spread","
        Appr"]
N = len(categories)
angles = [n/N * 2 * np.pi
          for n in range(N)]
angles += angles[:1]  # close
    polygon

fig, ax = plt.subplots(
    subplot_kw=dict(polar=True))

for profile, color, label in [
    (vals1, "#1565C0", "Tier 1"),
    (vals2, "#E53935", "Tier 5")
        ]:
    values = profile + profile
        [:1]
    ax.fill(angles, values,
        alpha=0.1, color=color)
    ax.plot(angles, values, "-o",
        color=color, lw=2,
        label=label)
```

Choosing the Right Chart

Parallel Coordinates vs Radar vs Pairs

Parallel Coordinates vs Radar vs Pairs Plot

Chart	Best For	Encoding	Caveat
Parallel coordinates	5–20 variables, many observations	Colour = group, highlight = accent	Axis order matters; reorder to reveal patterns
Radar / spider	3–8 variables, ≤3 profiles	Area = overall performance	Misleading area; axis order arbitrary
Pairs plot (SPLOM)	3–8 variables	Every pair shown; diagonal = marginal	$O(p^2)$ panels; scales poorly beyond 8

Decision Rules

Question	Answer	Chart
How many profiles to compare?	2–3	Radar
How many profiles?	Many (100s+)	Parallel coordinates
How many variables?	3–8	Radar or pairs
How many variables?	8–20	Parallel coordinates
Need precise value reading?	Yes	Pairs plot (scatter)
Need holistic "shape" comparison?	Yes	Radar
Need group separation?	Yes	Parallel coords + colour

Data Insight

In practice, **parallel coordinates** are far more common in data science than radar charts. Radar charts are popular in sports analytics (player profiles) and product comparisons but are statistically problematic. Default to parallel coordinates unless you have exactly 2–3 profiles for a general audience.

Interactive Extensions

Interactive Parallel Coordinates

```r
# R: plotly (brushable axes)
library(plotly)
plot_ly(ecar, type = "parcoords",
  line = list(color = ~Tier,
    colorscale = "Viridis"),
  dimensions = list(
    list(label = "FICO", values = ~FICO),
    list(label = "Rate", values = ~Rate),
    list(label = "Amount", values = ~Amount),
    list(label = "Term", values = ~Term),
    list(label = "Spread", values = ~Spread)))
```

```python
# Python: plotly
import plotly.express as px
fig = px.parallel_coordinates(
    ecar, color="Tier",
    dimensions=["FICO","Rate","Amount","Term","Spread"],
    color_continuous_scale="Viridis")
fig.show()  # interactive: brush each axis to filter
```

Pro-Tip

Interactive parallel coordinates (plotly) allow **brushing**: drag on any axis to select a range, and all lines outside that range fade. This is the most powerful exploratory use of parallel coordinates — impossible in static

Synthesis

Key Takeaways

1. **Parallel coordinates**: each vertical axis = one variable; lines connect one observation across all axes. Best for 5–20 variables, many observations.
2. **Reading patterns**: parallel = positive correlation, crossing = negative, cluster = group, stray line = outlier.
3. **Axis order is critical**: place correlated pairs adjacent. Try `ggparcoord(order="anyClass")` for algorithmic reordering.
4. **Grey+accent** prevents spaghetti: draw all lines grey, highlight one group in colour.
5. **Radar charts** work only for $\leq$3 profiles, $\leq$8 axes. Area is misleading; axis order is arbitrary.
6. **Interactive** parallel coordinates (plotly) with axis brushing is the most powerful multivariate exploration tool available.

Essential Readings

- Inselberg, A. (2009). *Parallel Coordinates: Visual Multidimensional Geometry*. Springer.
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 6 (Parallel Coordinates).
- Few, S. (2006). "Radar Graphs: Not So Useful After All." Visual Business Intelligence Newsletter.
- plotly documentation: `https://plotly.com/r/parallel-coordinates-plot/`

Next Module: Dimensionality Reduction Visualization — PCA, t-SNE, UMAP (W05-M03)